

ABISHEK DEVARAJ

192512126

Experiment No. 6

Threat Modeling and Security Hardening for an IoE Smart Banking System — Google Colab Simulation

AIM

To simulate threat modeling, user authentication, smart ATM transaction monitoring, cyberattack detection, and security hardening for an IoE-based Smart Banking System using Python in Google Colab.

OBJECTIVES

- Simulate secure user authentication.
- Generate smart ATM transactions.
- Identify potential cybersecurity threats.
- Detect ATM spoofing attacks.
- Detect brute-force attacks.
- Detect data-tampering attacks.
- Detect DoS attacks.
- Block suspicious transactions.
- Analyze transaction security status.
- Visualize the security results.
- Demonstrate security hardening in an IoE banking environment.

SOFTWARE REQUIRED

- Google Colab
- Python 3
- pandas
- matplotlib
- hashlib
- random

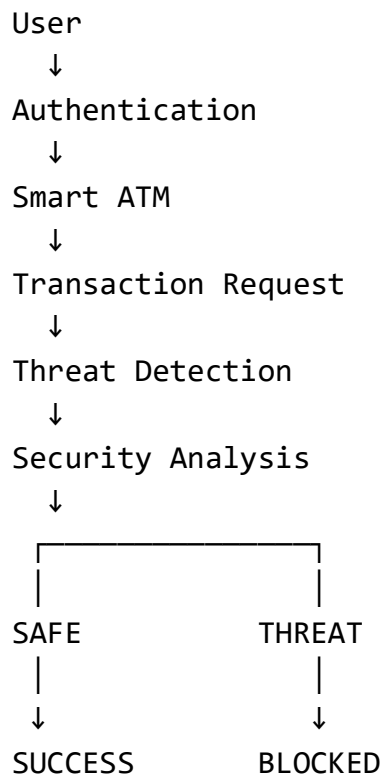
THEORY

The **Internet of Everything (IoE)** connects banking devices, users, data, processes, and cloud services through communication networks.

In a smart banking environment, devices such as ATMs continuously communicate with banking servers to process transactions. Since these devices are connected to networks, they may be exposed to cybersecurity threats.

Threat modeling is used to identify possible attacks and determine appropriate security actions.

The typical IoE banking security flow is:



In this experiment, Google Colab simulates:

- User authentication
- Password hashing
- Smart ATM transactions
- Threat identification
- Suspicious transaction blocking
- Security analysis

- Security visualization

ALGORITHM

1. Start the program.
2. Import the required Python libraries.
3. Create a banking user account.
4. Hash the user's password using SHA-256.
5. Authenticate the user.
6. Create smart ATM devices.
7. Generate customer transactions.
8. Generate transaction amounts.
9. Simulate possible security threats.
10. Classify transactions as SAFE or THREAT.
11. Block suspicious transactions.
12. Store transaction records in a DataFrame.
13. Count safe and blocked transactions.
14. Count individual threat types.
15. Display the security report.
16. Generate a security-status graph.
17. Display the final security result.
18. Stop the program.

GOOGLE COLAB CODE:

Experiment No. 6

Threat Modeling & Security Hardening

IoE Smart Banking System

Google Colab Simulation

```
import random import hashlib import pandas as pd import matplotlib.pyplot as plt

print("=" * 60) print(" IoE SMART BANKING SECURITY SYSTEM") print("=" * 60)
```

1. USER AUTHENTICATION

```
username = "admin" password = "Bank@123"

password_hash = hashlib.sha256( password.encode() ).hexdigest()

print("\n===== USER AUTHENTICATION =====")

print("Login Successful") print("Username :", username) print("Password Hash :",
password_hash[:20], "...")
```

2. SMART ATM TRANSACTIONS

```
atm_ids = [ "ATM001", "ATM002", "ATM003", "ATM004", "ATM005" ]

records = []
```

```
for atm in atm_ids:

    customer = random.randint(
        1001,
        1050
    )

    amount = random.randint(
        500,
        10000
    )

    threat = random.choice([

        "SAFE",

        "ATM Spoofing",

        "Brute Force",

        "Data Tampering",

        "DoS Attack"

    ])

    if threat == "SAFE":

        status = "SUCCESS"

    else:

        status = "BLOCKED"

    records.append([

        atm,

        customer,
```

```
        amount,  
        threat,  
        status  
    ])
```

3. DISPLAY TRANSACTION REPORT

```
df = pd.DataFrame(  
    records,  
    columns=[  
        "ATM ID",  
        "Customer ID",  
        "Amount",  
        "Threat",  
        "Status"  
    ]  
  
)  
  
print("\n==== ATM TRANSACTION REPORT =====")  
  
print(df.to_string(index=False))
```

4. SECURITY ANALYSIS

```
safe = len( df[df["Threat"] == "SAFE"] )

spooft = len( df[df["Threat"] == "ATM Spoofing"] )

brute = len( df[df["Threat"] == "Brute Force"] )

tamper = len( df[df["Threat"] == "Data Tampering"] )

dos = len( df[df["Threat"] == "DoS Attack"] )

success = len( df[df["Status"] == "SUCCESS"] )

blocked = len( df[df["Status"] == "BLOCKED"] )

print("\n===== SECURITY REPORT =====")

print( "Safe Transactions :", safe )

print( "ATM Spoofing :", spoof )

print( "Brute Force :", brute )

print( "Data Tampering :", tamper )

print( "DoS Attack :", dos )

print( "Successful :", success )

print( "Blocked :", blocked )
```

5. SECURITY HARDENING

```
df["Security_Action"] = df["Status"].apply(

lambda x:
"ALLOW TRANSACTION"
if x == "SUCCESS"
else
"BLOCK TRANSACTION"

)

print("\n===== SECURITY HARDENING =====")

for index, row in df.iterrows():

print(
    row["ATM ID"],
    "->",
    row["Threat"],
    "->",
    row["Security_Action"]
)
```

6. SECURITY STATUS GRAPH

```
plt.figure( figsize=(8, 5) )

bars = plt.bar(
```



```

["SAFE / SUCCESS",
 "THREAT / BLOCKED"],

[success, blocked]

)

plt.title( "IoE Smart Banking Security Status", fontsize=15, fontweight="bold" )

plt.xlabel( "Transaction Security Status" )

plt.ylabel( "Number of Transactions" )

plt.grid( axis="y", linestyle="--", alpha=0.5 )

```

Display values above bars

```

for bar in bars:

    height = bar.get_height()

    plt.text(

        bar.get_x()
        + bar.get_width() / 2,

        height + 0.05,

        str(int(height)),

        ha="center",

        fontweight="bold"

    )

plt.tight_layout()

plt.show()

```

7. SAVE SECURITY REPORT

```
df.to_csv(
    "ioe_smart_banking_security_report.csv",
    index=False

)

print( "\nSecurity report successfully saved as" )

print( "ioe_smart_banking_security_report.csv" )

print("\n" + "=" * 60)

print( "Threat Modeling Completed Successfully" )

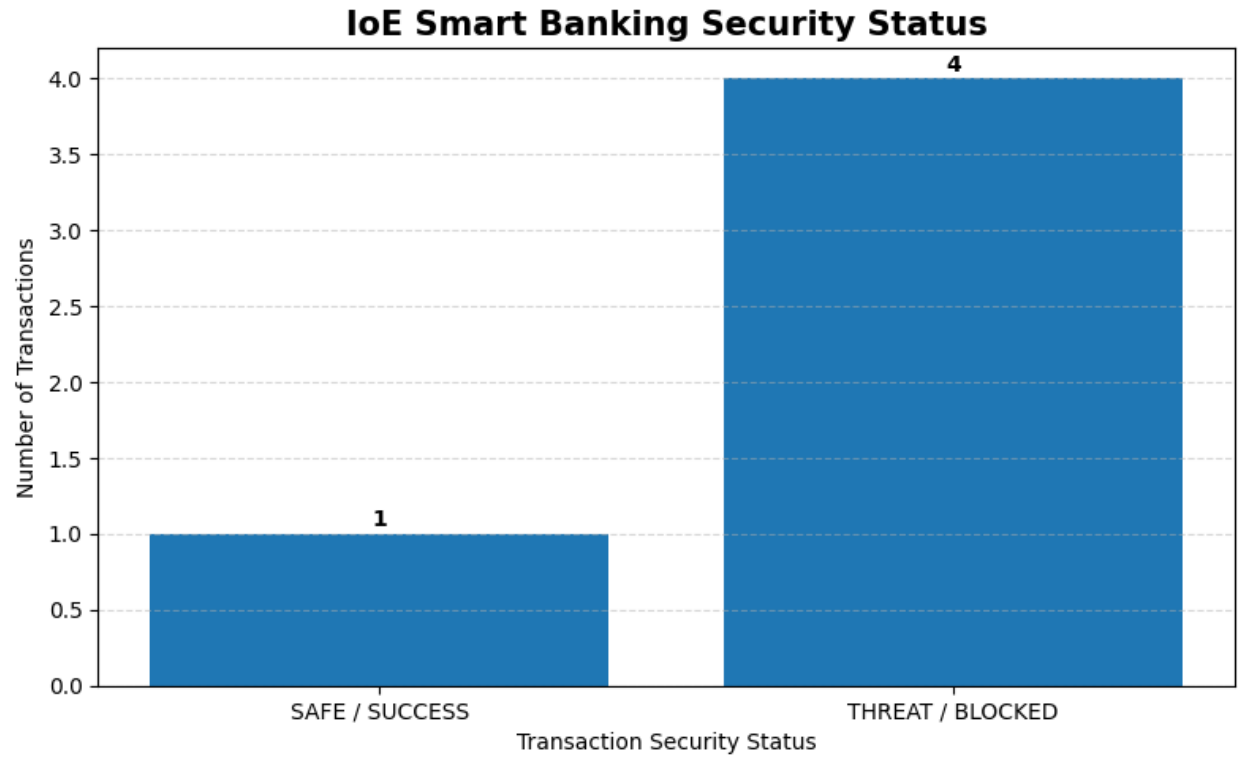
print( "Security Hardening Applied" )

print( "Suspicious Transactions Blocked" )

print( "IoE Banking System Protected Successfully" )

print("=" * 60)
```

OUTPUT:



RESULT:

The **Threat Modeling and Security Hardening for an IoE Smart Banking System** was successfully simulated using Python in Google Colab. User authentication was performed using SHA-256 password hashing, smart ATM transactions were generated, potential cybersecurity threats were identified, and suspicious transactions were automatically blocked. The security status was visualized using a graph, and the complete security report was successfully stored in CSV format. Thus, the experiment demonstrated **IoE-based authentication, threat modeling, attack detection, security hardening, and secure transaction management**.